

SEND5232: Software Architecture and Design Project

Bereket Kiros – ugr/189518/16 – Section 2

Phase 6 — Implementation Prototype / Demonstration

 Lihq'et Bookstore System

1. Introduction

The Lihq'et Bookstore System prototype is a functional full-stack web application that demonstrates the interaction between multiple system components, including the frontend interface, backend API, and database. The implementation validates the proposed architecture by showing real data flow, API communication, and persistent data storage.

2. System Implementation Overview

The system is implemented using the following technologies:

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Node.js with Express.js
- **Database:** MySQL

The application follows a RESTful architecture where the frontend communicates with the backend through HTTP requests, and the backend interacts with the database using SQL queries.

3. Implemented Modules

The prototype demonstrates interaction between at least three core modules:

1. Frontend Module

- Displays book data in a dynamic table
 - Provides a form to add and update books
 - Includes Edit and Delete buttons for each record
 - Uses Fetch API to communicate with backend
-

2. Backend Module

- Implements REST API endpoints:
 - GET /books → retrieve all books
 - POST /books → add a new book
 - PUT /books/:id → update book details
 - DELETE /books/:id → delete a book
 - Uses Controller, Service Layer, and Model structure
-

3. Database Module

- MySQL database with a **books table**
 - Stores:
 - id (Primary Key)
 - title
 - author
 - price
 - stock
 - Handles persistent storage and retrieval
-

4. System Interaction Demonstration

The prototype demonstrates the following workflow:

1. User enters book data in the frontend form
2. Frontend sends a POST request to the backend
3. Backend Controller receives the request
4. Service Layer validates and processes data
5. Model executes SQL query on MySQL
6. Database stores the record
7. Backend returns response
8. Frontend updates the table dynamically

This confirms successful interaction between all system layers.

5. Key Functional Features

The system successfully implements:

- Add new books
 - View all books
 - Update existing book details
 - Delete books
 - Dynamic UI updates without page reload
-

6. Evidence of Functionality

The prototype provides clear evidence of system functionality:

- Real-time updates in the UI after CRUD operations
- Successful API communication using HTTP methods
- Persistent data storage in MySQL database
- Proper request-response cycle using JSON format

The implementation of the Lihq'et Bookstore System is available through a public GitHub repository, which contains the full source code for both frontend and backend components.

- **GitHub Repository Link:**
<https://github.com/MrBekK/Lihqet-BookStore>

The repository includes:

- Backend implementation using Node.js and Express
- Frontend interface using HTML, CSS, and JavaScript
- RESTful API routes for CRUD operations
- Database integration with MySQL

The codebase demonstrates complete interaction between system components, including API communication, data processing, and persistent storage.

7. Architectural Validation

The implementation validates the designed architecture:

- The frontend interacts only with the Controller
- The Controller delegates logic to the Service Layer
- The Service Layer manages business rules
- The Model handles database operations

This confirms that the system follows the intended **MVC + Service Layer architecture**.

8. Conclusion

The developed prototype successfully demonstrates a working implementation of the Lihq'et Bookstore System. It verifies that the architectural design is practical and functional, showing effective interaction between frontend, backend, and database components. The system meets the project requirement by providing a functional demonstration of core modules and data flow.